

Programming Assignment – Written Report

Track B – Ray Tracing

Hannah Effat Manesh Nik (25809221)

A) What I built

Algorithm Implemented

I implemented a ray tracing algorithm, specifically the code from the first 9 chapters in Peter Shirley's *Ray Tracing in One Weekend* book, which is free to access online:

<https://raytracing.github.io/books/RayTracingInOneWeekend.html>

Language

The implementation uses C++20. I chose this because the algorithm is given in C++ in the book, and I didn't believe my understanding of the details of algorithm was good enough to implement in a different language. However, I struggled to implement clean C++ code, as my unfamiliarity with C++, as well as programming larger programs, proved to be difficult.

Data Representation

When the program is launched, a **SceneData** object is instantiated. This contains a vector of all Scenes that the user has saved in the Data/Scenes/ directory during launch. This is updated while the program is running, but only in under certain circumstances.

Scenes are a collection of variables that the user is allowed to modify. This includes the image width, colours, as well as objects within a scene and their positions. Each scene also contains its own Camera object.

Cameras are at the forefront of the ray tracing algorithm and brings each individual aspect of the algorithm together. On instantiation, it calculates image height, the spaces between pixels and sets all members meant to be modifiable by the user to the correct values. This may seem costly, but cameras are only instantiated right before rendering.

Vec3 (vector three) is a staple class in the ray tracing algorithm. It is represented by an array of three doubles. There are also two aliases that refer to Vec3, **Point3** and **Colour**. Although the function remains the same, Vec3s generally refer to directions, Point3s refer to locations and Colours refer to RGB colours.

Ray is a class that uses both Vec3 and Point3, represented by a direction and an origin. Given a parameter t , the point along the ray is calculated.

Hittable is a parent class to all objects (geometric shapes that are rendered) that may appear in a scene and represents anything a ray can intersect. It defines an abstract method Hit(...), which calculates whether or not the object has been hit by a ray.

Sphere is a child of **Hittable**. It is represented using a center point and radius. The center is represented as a three-dimensional point, while the radius is stored as a double. It defines its own Hit function based off these variables to determine if a ray has hit the object or not.

HittableList is a collection of **Hittables** that defines its own Hit function to find if any object the list has been hit by a ray. This allows any object derived from **Hittable** to be rendered in the same scene while the renderer can interact with them through a common interface.

Scenes are persisted using plain text files (.txt). Each scene contains rendering configurations followed by object properties. Plain text was chosen because it's simple to generate and parse and remains human-readable.

ErrorType is an enumeration containing the different error conditions that can be handled by the program. Its values are used to identify the cause of an error and generate an error message for the CLI.

Result was developed as a result type that represents either a successful operation or a failure containing an **ErrorType**, which describes the cause of the error. This avoids relying on exceptions and provides the CLI with enough context to display clear and informative error messages to the user.

Simplifications or Extensions

The modification to the ray tracing implementation in *Ray Tracing in One Weekend* were relatively minimal, as the main focus of this project was extending the renderer with a command line scene editor. The most noticeable changes I made were:

- The render() function, which was initially contained in camera.hpp, was moved to file_manager.hpp. This separated the rendering process from the camera's responsibilities and allowed the rendered output to be written to .ppm files as part of the file management functionality.
- To support the file_manager.hpp render function, getters were created for all private members of camera.hpp.
- The access modifiers of the Camera class were changed so that properties such as image dimensions, aspect ratio, anti-aliasing, and maximum ray depth could be modified by the scene editor.
- Methods and functionalities from the original implementation that were not required by my project were removed to reduce unnecessary code.
- A command line interface and scene management system were added around the ray tracing implementation. This allows users to create, modify and delete objects and scene properties before rendering.

- A Result type and ErrorType enumeration were added to handle errors from scene management and file i/o operations and provide informative feedback through the CLI.

Navigating the Code

While the program is running, there is an in-memory database in the form of the SceneData class. The SceneData class holds all current saved scenes and handles scene modifications in the CLI. The program also support persistent storage locally through scene.txt files. When a scene is modified, Scene files are updated within SceneData and are then saved to storage on disk. This is done to reduce file i/o in the program.

The entry point of the program is main.cpp. It attempts to load all scene files from the Data/Scenes/ directory into an instance of the SceneData class. If file_manager.hpp attempts to read incorrect file formatting, it will result in closure of the program. If SceneData was successfully created, main.cpp will find and run the CLI from cli.cpp.

Within CLI, all user inputs are handled. The class will compare the user input to all possible commands, and act accordingly. If the user input doesn't match any commands, an error will appear, prompting the user to use the 'help' command.

Creating, modifying or deleting a scene will have at least two main steps: applying the modification in the SceneData object, or Scene object, and then saving those changes to the user's local files.

If a user wants to create or delete a scene, the Scene object is created/delete in the SceneData instance. If that operation was successful, the corresponding scene file will also be created/deleted.

The ray tracing algorithms is contained within include/raytracing/, only accessible through Scene objects, with Scene.cpp acting as an interface between the CLI and underlying renderer. When a scene is rendered, Scene configures the camera and initiates the rendering process. The camera generates rays, which are tested against the scene's HittableList. Individual objects such spheres determine intersections, and return a HitRecord, which is then used to calculate the colour of the ray. The resulting pixel colours are written to .ppm file, which can be converted to a different image file format for viewing.

B) Track-Specific Writeup

What The Tool Does

This program aims to allow the intuitive creation, modification, deletion and storage of multiple scenes, or worlds, that can be rendered at will by the user. Currently, users can modify the skybox colour, and add objects, moving them or changing their sizes. The dimensions of the rendered image can also be modified, such as the aspect ratio and image width. Furthermore, render settings can be configured, affecting anti-aliasing and indirect lighting intensity.

How The Algorithm Sits Inside It

The CLI aims to provide the user with a way to configure the input to the algorithm, as well as request its execution. The Scene class sits between the CLI and the functionality of the ray tracer algorithm.

A Scene contains members that are configurable by the user. When a scene is rendered, the Scene uses these members to instantiate a Camera object, which handles ray generation based on the image dimensions, aspect ratio, samples per pixel (anti-aliasing) and maximum depth of ray bounces.

Rays then intersect with Hittables within an object's HittableList, creating a HitRecord and calculating the pixel's colour. The calculated colour is then converted to an RGB value in range $[0, 255]$, and written into an outputted .ppm file which can be converted to other viewable image formats.

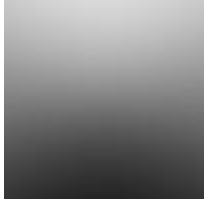
The components of the ray tracing algorithm are only accessed during the execution of the rendering, as a new Camera is instantiated at each execution, whether the scene has been modified or not.

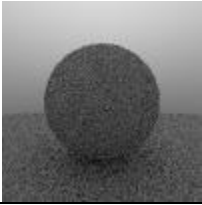
Interface Decisions

I decided to create a command driven interface because it seemed simple to implement, while allowing users to modify scenes without requiring a more complex interactive interface. This approach required relatively little code compared to a prompt-based system or more interactive CLI and allows for easy error handling due to strict command validation. Although, a limitation of this design is that commands must be entered using the correct syntax, placing more responsibility on the user than a prompt-based interface would.

Additionally, there are some commands that I didn't include in the 'help' outputs, such as 'render' while editing a scene to render that scene. I felt these were intuitive commands and didn't want to clutter the 'help' text.

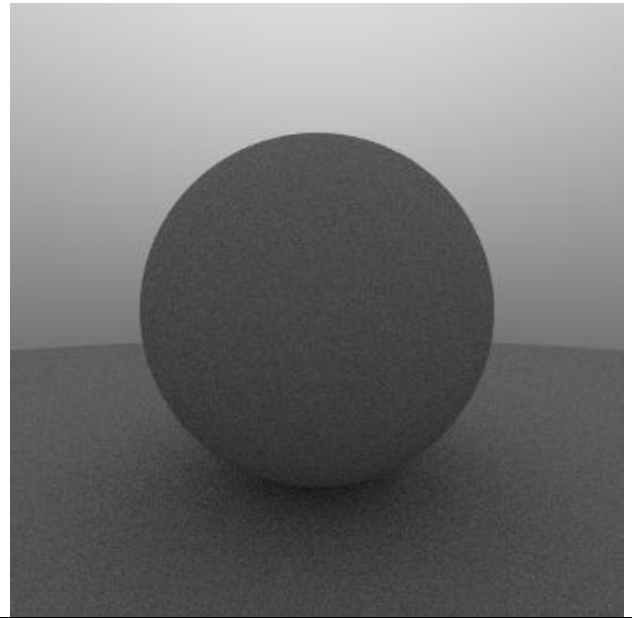
Worked Example

Command	Notations
<pre>> list No scenes saved. > </pre>	List all saved scenes.
<pre>> help commands: new <name> create a new scene edit <name> edit a scene list list scenes show <name> display scene contents render <name> render a scene delete <name> delete a scene quit exit program</pre>	View commands.
<pre>> new sample scene sample scene successfully create.</pre>	Creating a new scene called "sample scene". This also creates a new scene.txt file called "sample scene.txt".
<pre>> show sample scene scene_name: sample scene aspect_ratio: 1 image_width: 100 anti_aliasing: 10 max_depth: 10 skybox_colour_i: 0 0 0 skybox_colour_j: 1 1 1 No objects in scene.</pre>	Show scene properties and objects. These are the default values of any newly created scene.
<pre>> render sample scene sample scene successfully rendered. find the .ppm file at Data/Renders/sample scene. ppm</pre>	Render sample scene. Afterwards, converting it to a .png using ImageMagick(Magick "sample scene.txt" sample.png), this is the output: 

<pre>> edit sample scene</pre>		Now we are editing sample scene.
<pre>sample scene > sample scene > help commands: new <object_type> <object_name> create a new object <object_name> <object_variable> <value> edit an object <scene variable> <value> edit scene settings list list objects and their values show <object_name> list an object and its values delete <object_name> delete an object quit exit scene editing view git readme for specific commands for value variance.</pre>		Show commands relevant to scene editing.
<pre>sample scene > new sphere moo moo successfully added to sample scene</pre>		Create a sphere object called “moo” tin sample scene.
<pre>sample scene > show moo object_type: sphere object_name: moo center: 0 0 -1 radius: 0.5</pre>		Show the properties of moo. This is the default position and radius of any newly created sphere.
<pre>sample scene > new sphere floor floor successfully added to sample scene sample scene > floor center 0 -100.5 -1 Successfully modified floor sample scene > floor radius 100 Successfully modified floor</pre>		Create a sphere object called “floor” to sample scene. Modify the position of floor, so its center is at (0, -100.5, -1). Modify the radius of floor to be 100.
<pre>sample scene > render sample scene successfully rendered. find the .ppm file at Data/Renders/sample scene.ppm</pre>		Convert the sample scene.ppm to .png using Magick. sample1:

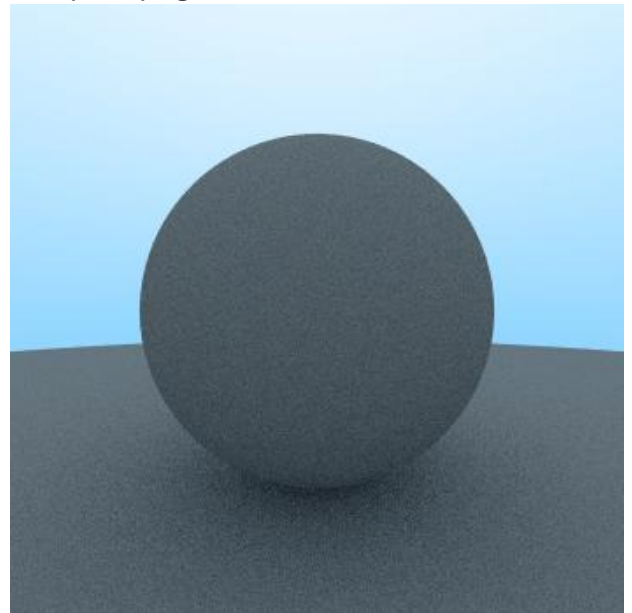
```
sample scene > image_width 350
image_width successfully configured.
sample scene > anti_aliasing 45
anti_aliasing successfully configured.
sample scene > max_depth 20
max_depth successfully configured.
sample scene > render
sample scene successfully rendered.
find the .ppm file at Data/Renders/sample scene.ppm
```

Modify scene properties and convert to .png using Magick.
sample2.png:



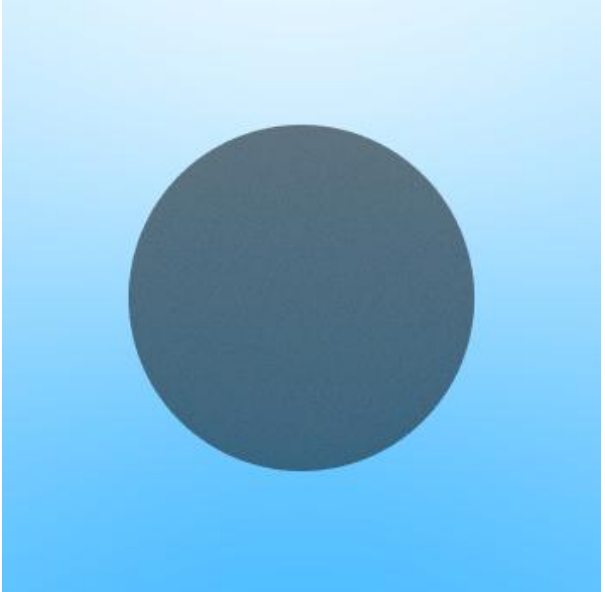
```
sample scene > skybox_colour_i 0.2 0.7 1
skybox_colour_i successfully configured.
sample scene > render
sample scene successfully rendered.
find the .ppm file at Data/Renders/sample scene.ppm
```

Change skybox_colour_i (bottom of world) scene property, and convert to .png using Magick. Note that given colour values must be in range [0, 1].
sample3.png:



```
sample scene > list
sphere moo (0, 0, -1) 0.5
sphere floor (0, -100.5, -1) 100
```

Listing all scene objects and their properties in a shorthand fashion.

<pre>sample scene > delete floor floor successfully delete from sample scene sample scene > list sphere moo (0, 0, -1) 0.5 sample scene > render sample scene successfully rendered. find the .ppm file at Data/Renders/sample scene.ppm</pre>	Deleting the sphere object “floor” in sample scene. Listing the remaining object. Rendering sample scene and using Magick to convert to .png. sample4.png: 
<pre>sample scene > quit No longer editing sample scene.</pre>	Quit editing. This is where sample scenes data will be saved on disk.
<pre>> quit </pre>	Quit the program. The application closes.

C) What I Learned

With barely any knowledge concerning ray tracing, and only genuine interest, I chose ray tracing for this assessment. It took me a long time to understand the presented concepts, and even longer to understand the code. Some of the vector math, and operations represented by the code are still mysteries to me and seem like black magic in making ray tracer work.

I was surprised at how much the ray tracer relies on vector math. It took me a while to understand the how the equation of a sphere could be converted to dot products, and quadratic equations. Using the discriminant to determine whether or not a ray has intersected with an object feels like it makes so much sense, but there was so much theory behind that one step. It felt really daunting when I first tried to understand it, and it still does.

When being introduced to new concepts in the first nine chapters of *Ray Tracing in One Weekend*, I initially struggled to understand what certain classes and functions were really doing. In chapter six, the Hittable class is introduced along with HitRecord. It took me a long time to understand why HitRecord was actually quite important, as it contains information concerning ray intersection which is used to calculate colour values for that pixel. In chapter nine, Peter Shirley implements a maximum number of times any ray can bounce between objects using a member “max_depth”. I really didn’t understand what this was doing and initially thought it was similar to the render distance that’s available in so many games. After a quick test, I knew how wrong I was. I was surprised that max_depth didn’t really affect the rendered image that much but now understand that it’s an optimization tactic.

As I’m not proficient with C++, and definitely didn’t use it optimally, I felt like I learned a bit about the language, such as implementing shared pointers, using and making my own header files, and file input and output. On a broader programming level, I definitely struggle with program architecture and implementing object-oriented principles. After spending some time refactoring my CLI code, I realised how important good architecture was, as it seemed so much easier to implement a feature I desired when classes had defined responsibilities.

D) AI Use

Before I started the CLI, I worked my way through the first nine chapters of *Ray Tracing in One Weekend* to ensure I understood what I could do with the algorithm Peter Shirley implements. I didn't understand the extent of what could be done, and during this time, I used ChatGPT as a private tutor of sorts. If I didn't understand a concept or piece of code (which happened often) I would tell GPT the context, my current understanding of the concept, and ask it to verify/strengthen my understanding. In this case, I don't believe GPT was misleading in any way, as its outputs made sense and I made a genuine effort to understand the content, asking follow-up questions when I didn't understand.

I used GPT heavily during implementation, primarily for debugging. As a CLI isn't necessarily an advanced program, I didn't have issues looking at Stack Overflow forums for help with using C++ standard libraries and syntax. There were a couple times when it was unhelpful fixing bugs. I believe this is simply due to a lack of context and not having access to all the code.

An issue that frequently occurred before I changed the architecture of the program was redefinition errors. I accidentally included the same header file multiple times throughout the program. I sent my errors to GPT and it recommended changes to lines. These changes didn't work, and I sent GPT the new errors. This continued for a while before I decided to fix it myself. I wrote down where each header was declared and eventually was able to fix the issue by modifying where headers were locating, slightly modifying the original ray tracing implementation given by Peter Shirley.

Another time, I asked GPT about a logic error in my code. My program was outputting both error and success messages, and I had no idea why. I asked GPT why this might be happening, and it gave me a few solutions but none of them worked. Eventually I was fed up and looked through the code myself. I eventually realised the issue was in another file, which is why GPT couldn't pick up on it, and solved it myself in about 10 minutes.

I made an effort to understand most of what GPT was outputting as an attempt to make up for my lack of programming knowledge and experience. The aspects I didn't completely understand but still chose to implement included some of the ray tracing aspects from *Ray Tracing in One Weekend* and the few times I asked GPT about C++ syntax, specifically concerning formatted outputs to the console.